Assignment 09

Published | Assign To | Edit | ⋮

Learning Objectives

IAM Users, Roles & Policies

Add IAM roles & policies needed to meet the assignment objectives to the Terraform configuration.

Amazon Simple Notification Service Configuration

Add Amazon Simple Notification Service (Amazon SNS) topic creation to your Terraform configuration.

Set up GitHub Repository for Lambda Functions

1. **Create (https://help.github.com/articles/create-a-repo/)** a new private GitHub repository for a web application in the GitHub organization you created.
2. The GitHub repository name must be `serverless`.
3. Grant TAs access to your GitHub repository.
4. Update `README.md` in your repository. Your readme file must contain the following:
    1. Prerequisites for Building and Deploying Your Application Locally.
    2. Build and deploy instructions for the web application.
5. Fork the GitHub repository in your namespace. You will do all development work on your fork.
6. All code for serverless should be in this repository.
7. Add appropriate `.gitignore` to your repository. A collection of useful .gitignore templates can be found **here (https://github.com/github/gitignore)**.

Email Service & AWS Lambda

Authentication and Access Control for AWS Lambda

- Add **Authentication and Access Control for AWS Lambda (https://docs.aws.amazon.com/lambda/latest/dg/lambda-auth-and-access-control.html)** to your Terraform templates.

Web Application Updates

- When a new user account is created, post a message on SNS topic with user details.
- Add a new endpoint in the web application to verify the user account from the link included in the email.

- If the UUID in the verification link is invalid of belongs to some other use, then it should not be accepted. The UUID should only be valid for 1 minute. If the user clicks on the link after the time has expired, the email address should not be validated.

Implement Lambda Function

1. Lambda function will be invoked by the SNS notification. The Lambda function is responsible for sending an email to the user.
2. As a user, I should receive an email when I create an account to confirm my email address. The link should be something like **http://URL/validateEmail?email=someone@example.com&token=<UUID>**
3. As a user, I should NOT receive duplicate email messages.
4. Track emails being sent so that user does not get duplicate emails even if duplicate messages are posted to the SNS topic.

Implement CI/CD Pipeline for Lambda Function

Using the same principles as in previous assignments, implement continuous deployment for Lambda functions. Every commit to the `serverless` repository should trigger build and deployment of your updates function to AWS Lambda.

CI/CD for Web Application - Update Pull Request Merged Workflow

No changes are required for the Pull Request Raised Workflow. This assignment requires you to use both `Dev` and `Demo` account in the same GitHub Workflow. You will need to configure access keys and IDs for both accounts with unique names for the GitHub Actions secrets.

1. Update the GitHub actions workflow and configure it to trigger when a pull request is merged. This workflow should do the following:
    1. Run the unit test. (No changes from current CI implementation)
    2. Validate Packer Template (No changes from current CI implementation)
    3. Build Application Artifact(s) (No changes from current CI implementation)
    4. Build AMI in DEV AWS account(No changes from current CI implementation)
        1. Upgrade OS packages
        2. Install dependencies (python, node.js, etc.)
        3. Install application dependencies (pip install for Python)
        4. Set up the application by copying the application artifacts and the configuration files.
        5. Configure the application to start automatically when the VM is launched.
        6. Share the AMI with the DEMO AWS account.

5. [NEW] Reconfigure GitHub Runner's AWS CLI to use access keys for the DEMO AWS account.
6. [NEW] Create a new Launch Template version with the latest AMI ID for the autoscaling group. The autoscaling group should be configured to use the latest version of the Launch Template.
7. [NEW] Issue command to the auto-scale group to do an "instance refresh" using the AWS CLI.
8. [NEW] GitHub Actions workflow must wait for instance refresh to complete before exiting. The status of the GitHub Actions workflow must match the status of the "instance refresh" command.
9. You are not required to handle rollback for failed deployments.

AWS Key Management Service

1. Create AWS KMS keys in the region where you deploy your resources.
2. Create separate encryption keys for each of the following resources:
    1. EC2
    2. RDS
    3. S3 Buckets
    4. Secret Manager for (Database Password for RDS instance & Credentials for Email Service)
3. The KMS should be set up with `90 day` rotation period.
4. Use KMS keys when creating resources in Terraform.

Database Password

The database password auto-generated by the Terraform template must now be stored in Secret Manager with a custom KMS key with custom and retrieved by the user-data script to configure the web application.

Secure your web application endpoints with valid SSL certificates.

- For `dev` environment, you may use the AWS Certificate Manager service to get SSL certificates.
- For `demo` environment, you must request an SSL certificate from Namecheap or any other SSL vendor except for AWS Certificate Manager, import it into AWS Certificate Manager from your CLI, and then configure your load balancer to use the imported certificate.
- The command to import the certificate must be documented in your README.md file.
- Plain text requests sent to HTTP do not have to be supported.
    - HTTP to HTTPS redirection is not required.
- Traffic from the load balancer to the EC2 instance can use plain text protocol such as HTTP.

- Users should not be able to connect to the EC2 instance directly.

Submission Requirements

Step 1: Create Submission Folder

Create a folder using this naming convention:

```
firstname_lastname_neuid_a_##
```

Replace `##` with the assignment number (e.g., `01`, `02`, etc.)

Step 2: Clone Repositories

1. Navigate to your submission folder
2. Clone **all** required GitHub repositories using:

bash

```
git clone [repository-url]
```

3. **Important:** Use `git clone` to preserve your complete commit history and branch information

Step 3: Create and Verify ZIP File

1. Create a ZIP file of your submission folder:

```
firstname_lastname_neuid_a_##.zip
```

2. **Verification Step:** Extract your ZIP file in a different location and confirm:
   - All repositories are present
   - Commit history is intact
   - All required branches exist

Step 4: Submit

Upload your ZIP file to the assignment submission portal.

Important Notes

- **Multiple Submissions Allowed:** You may resubmit if needed. Only your latest submission will be graded.
- **Quality Check:** Always verify your ZIP file contents before final submission to avoid issues.
- **Complete Repositories Required:** Submit code from all repositories mentioned in the assignment requirements.

Troubleshooting

If you encounter issues with your ZIP file, feel free to resubmit after verification. Take time to ensure all requirements are met before your final submission.

Grading Guidelines for Cloud-Native Web Application

System Stability & Performance

- **No Server Errors**: Application must not generate 500 internal server errors during testing
- **Application Uptime**: No restart of the application server required between API calls
- **Error Handling**: Application should handle invalid requests gracefully with appropriate HTTP status codes

Demo Requirements

- **Platform**: Students must demonstrate the web application from their own laptops
- **API Testing**: APIs must be tested using REST clients (Postman, Restlet, Insomnia, etc.)
  - Browser-based testing is **not acceptable**
- **UI Restriction**: Application must **not** have any user interface components
  - Verify complete absence of HTML pages, forms, or visual components

Security Implementation

Authentication & Authorization

- **Authentication Method**: Must use Basic Authentication with a token-based approach
  - Session-based authentication is **not allowed**
  - JWT tokens are **not allowed**
- **Password Security**: All passwords must be encrypted using BCrypt with salt
  - Verify in the database that passwords are properly hashed
  - Check that salt is unique per password

Data Protection

- **Response Payload Security**: Password fields must **never** be included in API responses
  - Test all endpoints that return user data
- **Username Validation**: Only valid email addresses can be used as usernames
  - Test rejection of non-email username formats

Data Integrity & Business Logic

Account Management

- **Duplicate Prevention**: Application must prevent the creation of multiple accounts with the same email address
  - Test duplicate email registration scenarios

    - Verify appropriate error responses for duplicate attempts
- **Audit Fields Protection**: System-managed fields must be read-only
    - `account_created` field: Set automatically by application, never by user input
    - `account_updated` field: Set automatically by application, never by user input
    - Test attempts to manually set these values through API calls

Code Quality & Standards

Configuration Management

- **No Hard-Coding**: Variable values must be configurable (environment variables, config files)
- **Default Values**: Acceptable when overrideable during runtime or demos
- **Demo Flexibility**: Values must be modifiable when requested by the instructor/TA

Code Style & Documentation

- Follow industry-standard style guides for your programming language
- Include meaningful comments for complex logic
- Maintain consistent formatting and naming conventions
- Provide clear README files with setup and usage instructions

Application Stability & Error Handling

Runtime Stability

- **No Crashes**: Application must handle API calls without crashing
- **No Restarts**: Application/VM should not require restarts during demos (unless specifically required for feature demonstration)
- **Graceful Degradation**: Handle edge cases and invalid inputs appropriately

Error Management

- **No 500 Errors**: All server errors must be properly handled
- **Meaningful Error Messages**: Provide clear, actionable error responses
- **Input Validation**: Validate and sanitize all user inputs
- **Resource Management**: Properly handle database connections, file operations, and external API calls

Edge Case Handling

- Test boundary conditions (empty inputs, maximum values, null cases)
- Handle network timeouts and connection failures
- Implement appropriate fallback mechanisms
- Validate data integrity throughout the application lifecycle

Repository Content & Hygiene

File Exclusion Standards

- **IDE Files**: No IDE-specific configuration files (`.idea/` , `.vscode/` , `.eclipse/` , etc.)
- **Build Artifacts**: No compiled files or build outputs
  - Java: No `.class` , `.jar` , `.war` files
  - Node.js: No `node_modules/` directory
  - General: No `build/` , `dist/` , `target/` directories
- **System Files**: No OS-specific files (`.DS_Store` , `Thumbs.db` , etc.)

Dependency Management

- **External Dependencies**: Third-party libraries from package managers (Maven, npm, pip) must **not** be committed
- **Gitignore Configuration**: Verify `.gitignore` file is properly configured and comprehensive
  - Must include language-specific patterns
  - Must exclude build artifacts and dependencies
  - Should include common IDE and OS files

Grading Guidelines

Warning: The following guidelines are for information only and are subject to change at the discretion of the instructor and TA.

Common Grading Rubrics

Git

1. All commits to GitHub repositories must be made via pull request. No changes can be made by pushing commits directly to the main branch. The exception to this is the first commit made to the repository to initialize it, which can be done directly from the UI.
2. All pull requests must have status checks and be enforced. Merging pull requests that did not pass status checks may result in penalties.

Security

1. Committing any kind of credentials to the code or uploading them as part of your Canvas submission may result in a penalty.
2. Service providers may automatically disable your keys or suspend your account if they detect keys being committed in your GitHub repositories, even if they are private. This can significantly impact your assignment grade and your ability to complete assignments in the course. This is not grounds for assignment extensions; late penalties will apply as documented in the syllabus.

Software Engineering Best Practices

1. Values for variables should not be hardcoded in the code. It's OK to set default values for variables, but if the value cannot be modified during the demo or when the TA or instructor asks, you may be penalized.
2. Follow industry standards and use a style guide for your code. You can find them with simple web searches.

Code Stability, Error Handling, & Edge Cases

1. The application should not crash between various API calls.
2. The application, VM, or other resources should not require a restart during the demo unless the grading guidelines require it to demonstrate a specific feature/functionality.
3. The server cannot respond with a 500 Internal Server Error. This usually highlights areas of code that do not have proper error handling.

Previous Assignment Objectives

TAs must verify that students have resolved all open issues from the previous assignment(s).

Git & GitHub Repository (2%)

Git Repository Content Check

1. Check the repository for any IDE-specific files. IDE configuration files must not be in the repository.
2. Verify their .gitignore configuration.
3. Check the repository for build artifacts such as `.class`, `.jar`, `.war` files and `build`, `node_modules`, `venv` directory. None of these should be checked into the repository.
4. Check for dependencies. Dependencies from the Maven repository, npm, or Python should not be committed to the git repository.

Git Forking Workflow

1. No direct commits are made to the organization's repository. Verify that the student works from a forked repository and uses the feature branch. There should be no direct commits to the main branch in their forked repository.

Points	10
Submitting	a file upload
File Types	zip
Allowed Attempts	5

Due	For	Available from	Until
Nov 15 at 9pm	Everyone else	Nov 6 at 6pm	Nov 15 at 9pm
Nov 17 at 6pm	1 student	Nov 6 at 12am	Nov 17 at 6pm

+ Rubric